通过注解来初始化 Java Web 组件
及 Spring MVC 注解式初始化 Web 组件的分析

George <www.georgeinfo.com>|2018-4-2

1 前言

传统的 Java Web 应用，是靠 Web 容器（比如 tomcat）读取/WEB-INF/web.xml 文件，来得到 Servlet Context Listener、Filter 和 Servlet 等组件类的类路径和配置参数信息，从而按照一定的顺序初始化这些类，他们的初始化顺序是：

对于 Servlet Context Listener 类型的组件，或者 Filter 类型的组件，如果同类型组件声明了多个类，则这多个类按照在 web.xml 配置文件中，定义的先后顺序来初始化，比如：如果定义了多个 Servlet Context Listener 监听器类，则 Java Web 容器会根据他们在 web.xml 定义的顺序，自上而下来初始化，Filter 也是这样。

Servlet 比较特殊，如果 Servlet 在 web.xml 的配置中，声明了<load-on-startup>自动初始化的顺序</ load-on-startup>属性，则 Servlet 会在 Web 应用被容器扫描时（如 tomcat 启动时），多个自动初始化的 Servlet 会按照他们各自的<load-on-startup>属性大小来以此初始化。

如果 Servlet 没有声明<load-on-startup>属性，则默认 Servlet 是不会自动初始化的，它们会在用户第一次访问他们的 URI 路径时，触发初始化工作。

基本上，靠 Servlet Context Listener、Filter 和 Servlet 这三类 Java Web 组件，就基本构成了一个 Java Web 体系的运作。

传统上，这三类组件，都是写在/WEB-INF/web.xml 配置文件中的，不过自从 Servlet 3.0 规范开始（Servlet3.0 是 Java EE6 规范的一部分），可通过注解的方式来声明 Servlet Context

Listener、Filter 和 Servlet 这些组件，Web 容器在扫描 Java Web 应用的所有/WEB-INF/lib/*.jar 和/WEB-INF/classes 类路径里的类文件时，会判断每个类是否写有 Servlet 3+规范的注解，如果扫描到了，就视同跟 web.xml 中明文声明的组件一样来对待（当然，你也可以不用 web.xml，所有组件都通过注解的方式来声明）。

2 web.xml 方式显示声明 Web 组件

传统上，Web 组件的声明，通过/WEB-INF/web.xml 文件来进行，一个典型的 web.xml 配置内容如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app
version="3.0"
xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd">
	<display-name>Spring3MVC</display-name>
	<!-- Spring ApplicationContext 配置文件的路径,可使用通配符，用于后面的 Spring Context Loader -->
	<context-param>
		<param-name>contextConfigLocation</param-name>
		<param-value>classpath*:resources/spring/applicationContext-*.xml</param-value>
	</context-param>
	<!—设定 Spring Context 的默认 Profile -->
	<context-param>
		<param-name>spring.profiles.default</param-name>
		<param-value>production</param-value>
	</context-param>
	<context-param>
		<param-name>webAppRootKey</param-name>
		<param-value>webName.root</param-value>
	</context-param>
	<context-param>
		<param-name>log4jConfigLocation</param-name>
		<param-value>WEB-INF/log4j.properties</param-value>
	</context-param>
	<context-param>
		<param-name>log4jRefreshInterval</param-name>
		<param-value>3000</param-value>
	</context-param>
```

```
<listener>
    <!--Spring ApplicationContext 载入 -->
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<listener>
      <listener-class>
            org.springframework.web.util.Log4jConfigListener
      </listener-class>
</listener>


<filter-mapping>
      <filter-name>CharacterEncodingFilter</filter-name>
      <url-pattern>/gateway/*</url-pattern>
      <dispatcher>REQUEST</dispatcher>
      <dispatcher>FORWARD</dispatcher>
</filter-mapping>
<filter-mapping>
      <filter-name>CharacterEncodingFilter</filter-name>
      <url-pattern>/weixin/*</url-pattern>
      <dispatcher>REQUEST</dispatcher>
      <dispatcher>FORWARD</dispatcher>
</filter-mapping>

<filter-mapping>
      <filter-name>GeorgeFilter</filter-name>
      <url-pattern>/weixin/*</url-pattern>
      <dispatcher>REQUEST</dispatcher>
      <dispatcher>FORWARD</dispatcher>
      <dispatcher>INCLUDE</dispatcher>
      <dispatcher>ERROR</dispatcher>
</filter-mapping>


<filter>
      <!-- 字符编码转换器过滤器 -->
      <filter-name>CharacterEncodingFilter</filter-name>
      <filter-class>org.springframework.web.filter.CharacterEncodingFilter</filter-class>
      <init-param>
            <param-name>encoding</param-name>
            <param-value>utf-8</param-value>
      </init-param>
      <init-param>
```

```
                <param-name>forceEncoding</param-name>
                <param-value>true</param-value>
            </init-param>
        </filter>
        <filter>
            <filter-name>GeorgeFilter</filter-name>
            <filter-class>com.georgeinfo.springweb.env.GeorgeFilter </filter-class>
        </filter>

        <servlet>
            <servlet-name>springServlet</servlet-name>
            <servlet-class>com.georgeinfo.springweb.global.SpringWebServlet</servlet-class>
            <init-param>
                <param-name>contextConfigLocation</param-name>
                <param-value>classpath*:resources/spring/spring-mvc*.xml</param-value>
            </init-param>
            <init-param>
                <!-- 授权认证服务，是这个类来处理的 -->
                <param-name>BeforeSpringWebInitClassName</param-name>
              <param-value>
              com.georgeinfo.springweb.service.system.PrepareService
              </param-value>
            </init-param>
            <load-on-startup>1</load-on-startup>
        </servlet>

        <servlet-mapping>
            <servlet-name>springServlet</servlet-name>
            <url-pattern>/</url-pattern>
        </servlet-mapping>
        <mime-mapping>
            <extension>txt</extension>
            <mime-type>text/plain</mime-type>
        </mime-mapping>
        <session-config>
            <session-timeout>
                1339
            </session-timeout>
        </session-config>
        <welcome-file-list>
            <welcome-file>index.ghtml</welcome-file>
            <welcome-file>index.jsp</welcome-file>
        </welcome-file-list>
    </web-app>
```

其实，我个人还是觉得，通过 web.xml 配置文件，显示声明式初始化 Java Web 组件更好，这样整个系统中，所有的 Web 组件，入口，一目了然，无论实际的实现类分布在应用中的哪个 jar 包，哪个包路径下，都可以通过一个 web.xml 来一步步追踪，很方便。

不过不知道为什么，现在很流行通过注解的方式，已硬编码编程的方式，来声明 Web 组件类的实例，本文咱们就追随这个潮流，重点分析一下 Java Web 注解式初始化组件的过程。

3 Servlet 3.0+ 的注解方式声明 Web 组件

从 Servlet 3.0 开始，可以通过注解的方式来注册和初始化各种 Java Web 组件，也就是通过编程的方式，来声明 Web 组件。

Java Web 容器在加载 Web App 的时候（比如 Tomcat 启动时扫描 webapps），会扫描所有.jar 包和 classes 目录下的所有类，如果类添加了符合要求的 Java Web 组件注解，就会被识别为同 web.xml 中显示声明的组件一样，被初始化。

比如，如果 Web 容器扫描到类上面的**@WebServlet** 注解，会把该类识别为 Servlet 来初始化，如果扫描到**@WebFilter**，会识别为过滤器来初始化，如果扫描到**@WebListener**，会被识别为监听器：Request 监听器、Servlet Context 监听器或者 Session 监听器。

具体类型，根据**@WebListener** 注解所标注类，实现了哪个接口来决定：

- javax.servlet.ServletContextListener
- javax.servlet.ServletContextAttributeListener
- javax.servlet.ServletRequestListener
- javax.servlet.ServletRequestAttributeListener
- javax.servlet.http.HttpSessionListener
- javax.servlet.http.HttpSessionAttributeListener
- javax.servlet.http.HttpSessionIdListener

本文的重点是讲解注解注解式初始化 Java Web 组件的入口，所以就不详细解释上面这些注解的用法了，感兴趣的同学，可以去查看《Servlet 3.0/3.1 规范》文档。

一个典型的注解式 Servlet 类代码如下：

```
@WebServlet(name = "NewServlet", urlPatterns = {"/NewServlet"})
public class NewServlet extends HttpServlet {
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlet NewServlet</title>");
            out.println("</head>");
            out.println("<body>");
            out.println("<h1>Servlet NewServlet at " + request.getContextPath() + "</h1>");
            out.println("</body>");
            out.println("</html>");
        }
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }}
```

4 Spring MVC 注解式初始化 Java Web 组件的过程

4.1 前置知识解释之 Java SPI 机制

Java SPI 机制(Service Provider Interface)，是一种 Java 服务的发现、实例化和自动执行的插件机制，这种机制主要面向的是插件或框架平台的提供方，按照这种机制，Java 执行容器（比如 Tomcat）可以在众多 classes 类和.jar 包中的类之中，发现自己需要寻找的规定接口的实现类，并实例化这些实现类，并执行这些类约定的方法。

总之，SPI 就是一种服务的发现机制，简单来说就是一种约定文件名字的配置文件方式，方便让 Java 容器去寻找自己所需要的类，Java SPI 机制的运作过程如下：

Java 容器在加载 Java 应用的时候，会扫描每一个.jar 包和 classes 目录下的 *META-INF/service* 目录，如果这个目录中有一个以某接口名字（包括包路径的全名，比如：*javax.servlet.ServletContainerInitializer*）命名的文件，则容器会去读取这个文件，这个文件中会写一个实现类的全名（比如：*org.springframework.web.SpringServletContainerInitializer*），根据 Java 容器的不同，会对读取到的不同接口实现类做不同处理，比如 Java Web 容器，会对读取到的 *javax.servlet.ServletContainerInitializer* 接口实现类，进行实例化，并反射执行这个实现类的 *void onStartup(Set<Class<?>>* `classes`*, ServletContext context)*方法。

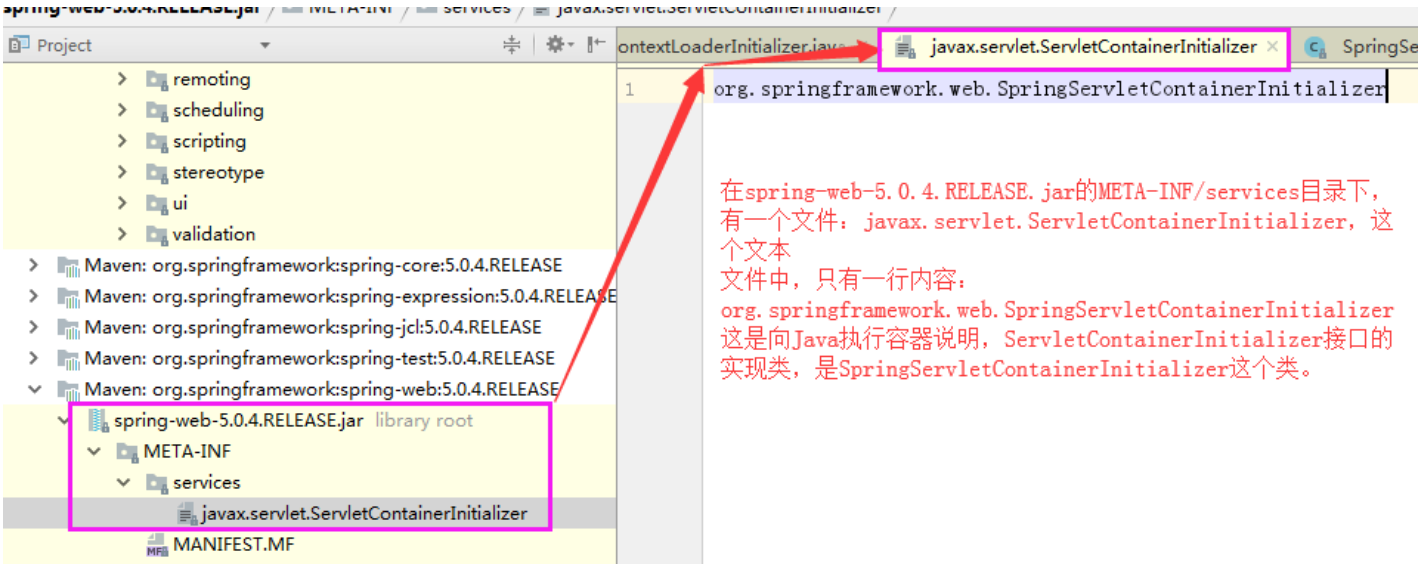
这种依靠 Java 规范约定，Java 执行容器按照规范去寻找、实例化和执行规定方法的机制制，就是 Java 的 SPI 机制。

4.2 Sping MVC 依靠 SPI 机制，进行 Web 组件初始化的过程分析

上面讲解了 Java 中的 SPI 机制以后，就方便理解 Spring MVC 的 Web 组件初始化过程了，以下 Spring 的初始化过程，以 Spring 5.0.4 作为例子来讲解。

Spring WEB 中的 ***org.springframework.web.SpringServletContainerInitializer*** 实现类，使用

的 SPI 配置内容如下图所示：



Spring MVC 的 ***org.springframework.web.SpringServletContainerInitializer*** 这个实现类，实现了 JavaEE Servlet 规范所规定的 ***javax.servlet.ServletContainerInitializer*** 接口，根据 Servlet 3.0 规范，Web 容器在加载 web 应用时，要扫描所有的.jar 包和 classes 类路径，按照 SPI 机制，寻找 ***ServletContainerInitializer*** 接口的实现类，一旦找到，就通过反射实例化这个实现类，然后动态执行它，触发它的方法：

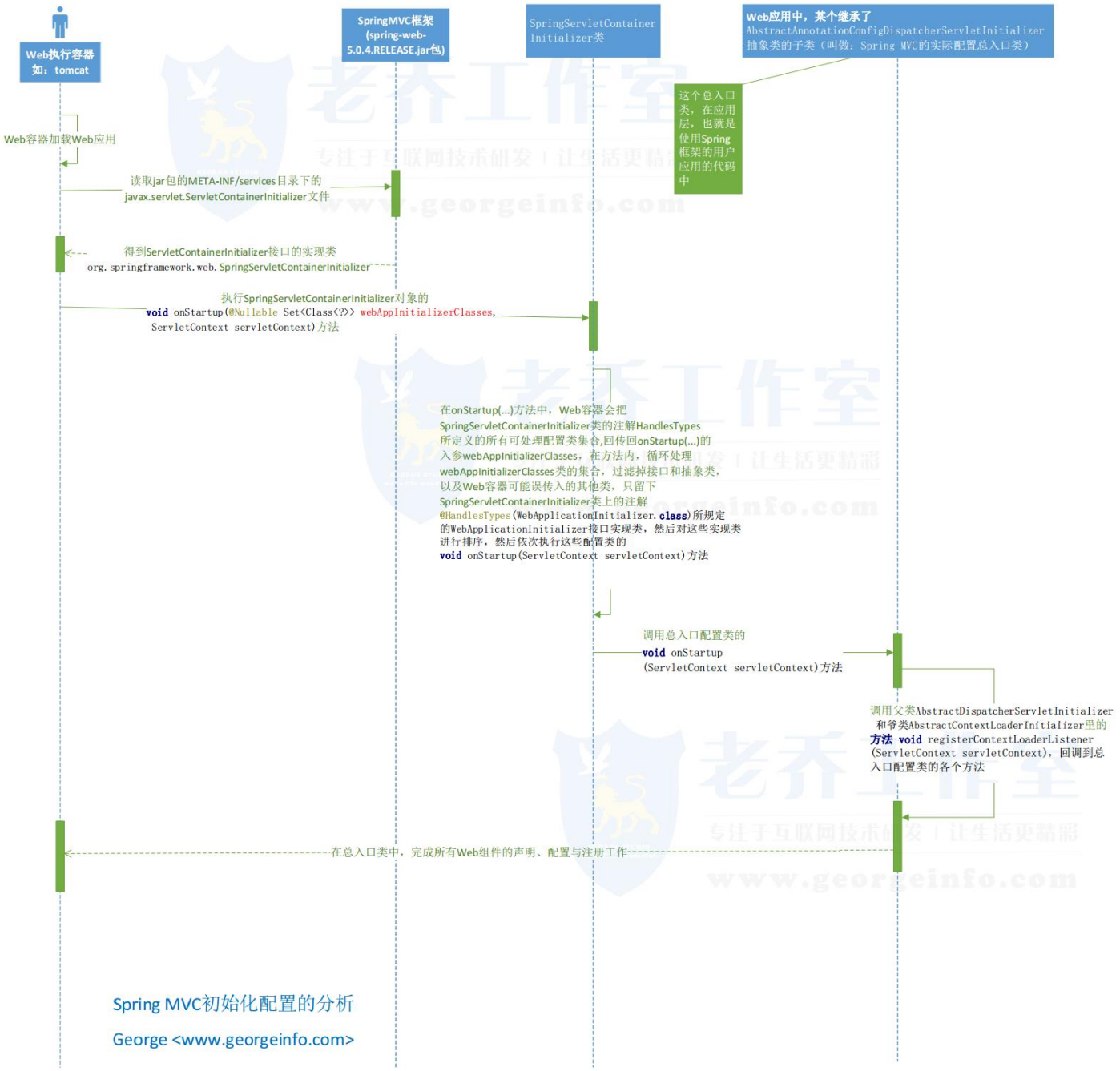
void onStartup(Set<Class<?>> `classes`*, ServletContext context)*

这个方法的 classes 参数，是 Web 容器扫描到的符合条件的类的集合（符合条件是指：*SpringServletContainerInitializer* 实现类上的注解@HandlesTypes(可以处理的配置类.class)，注解 value 所定义的配置类，以及这些配置类的所有子类），这些可以处理的配置类，会被 Java Web 容器扫描到以后，回传回 *org.springframework.web.SpringServletContainerInitializer* 这个实现类的 *void onStartup(Set<Class<?>>* `classes`*, ServletContext context)* 方法的入参中。

Spring MVC/Spring Boot，就是按照这个原理，在 onStartup(...)方法中，实现所有 Web 组件的注册和初始化过程，整个初始化过程如下：

Spring MVC初始化配置的分析

George <www.georgeinfo.com>



5 典型的 Spring MVC 应用 APP 配置类

以下代码取自 SpringFramework 的官方示例代码[1]，展示了简单而标准的 Spring MVC 配置类的内容，代码如下：

配置总入口类：***MvcShowcaseAppInitializer***

```
public class MvcShowcaseAppInitializer extends
AbstractAnnotationConfigDispatcherServletInitializer{

   @Override
   protected Class<?>[] getRootConfigClasses() {
      return new Class[] { RootConfig.class };
   }

   @Override
   protected Class<?>[] getServletConfigClasses() {
      return new Class[] { WebMvcConfig.class };
   }

   @Override
   protected String[] getServletMappings() {
      return new String[] { "/" };
   }

   @Override
   protected Filter[] getServletFilters() {
      return new Filter[] { new DelegatingFilterProxy("csrfFilter") };
   }

}
```

根配置类：***RootConfig***，主要是一些授权认证的配置

```
// Root Context: defines shared resources visible to all other web components
@Configuration
public class RootConfig {

   // CSRF protection. Here we only include the CsrfFilter instead of all
of Spring Security.
   // See
http://docs.spring.io/spring-security/site/docs/current/reference/htmlsi
```

[1] https://github.com/spring-projects/spring-mvc-showcase

```
ngle/#csrf
   // for more information on Spring Security's CSRF protection
   @Bean
   public CsrfFilter csrfFilter() {
      return new CsrfFilter(new HttpSessionCsrfTokenRepository());
   }

   // Provides automatic CSRF token inclusion when using Spring MVC Form tags
or Thymeleaf.
   // See http://localhost:8080/#forms and form.jsp for examples
   @Bean
   public RequestDataValueProcessor requestDataValueProcessor() {
      return new CsrfRequestDataValueProcessor();
   }
}
```

Spring MVC 的前置处理器配置：*WebMvcConfig*

```
// DispatcherServlet context: defines Spring MVC infrastructure
// and web application components
@Configuration
@ComponentScan(basePackages = "org.springframework.samples.mvc")
@EnableWebMvc
@EnableScheduling
public class WebMvcConfig implements WebMvcConfigurer {

   @Override
   public void addFormatters(FormatterRegistry registry) {
      registry.addFormatterForFieldAnnotation(new
MaskFormatAnnotationFormatterFactory());
   }

   @Override
   public void addArgumentResolvers(List<HandlerMethodArgumentResolver>
resolvers) {
      resolvers.add(new CustomArgumentResolver());
   }

   // Handle HTTP GET requests for /resources/** by efficiently serving
   // static resources under ${webappRoot}/resources/
   @Override
   public void addResourceHandlers(ResourceHandlerRegistry registry) {
```

```
registry.addResourceHandler("/resources/**").addResourceLocations("/reso
urces/");
    }

    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/").setViewName("home");
    }

    @Override
    public void configureViewResolvers(ViewResolverRegistry registry) {
        registry.jsp("/WEB-INF/views/", ".jsp");
    }

    @Override
    public void configurePathMatch(PathMatchConfigurer configurer) {
        UrlPathHelper pathHelper = new UrlPathHelper();
        pathHelper.setRemoveSemicolonContent(false); // For
@MatrixVariable's
        configurer.setUrlPathHelper(pathHelper);
    }

    @Override
    public void configureAsyncSupport(AsyncSupportConfigurer configurer) {
        configurer.setDefaultTimeout(3000);
        configurer.registerCallableInterceptors(new
TimeoutCallableProcessingInterceptor());
    }

    @Bean
    public MultipartResolver multipartResolver() {
        return new CommonsMultipartResolver();
    }

}
```

6 总结

Spring MVC 通过 Java 的 SPI 机制，通过读取 classes 及 jar 包的 META-INF/services 中的

接口配置文件，读取到 Servlet 容器所需要的 ***javax.servlet.ServletContainerInitializer*** 接口实现类，通过实例化该实现类，调用它的

void onStartup(Set<Class<?>> `classes`*, ServletContext context)* 方法，最终实现类对 Web 应用配置文件的读取与执行。